

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

10 - Immediate Operands

Constant

- Many times a program will use a constant in an operation
- For example, incrementing an index to point to the next element of an array
- Using only instructions we have seen so far, we would have to load a constant from memory to use one
- Constants would have been placed in memory when program was loaded

Example

- Add constant 4 to register \$s3. Assuming that \$s1 + AddrConstant4 is the memory address of the constant 4.
- We could use the code

Example (cont.)

- Add constant 4 to register \$s3. Assuming that \$s1 + AddrConstant4 is the memory address of the constant 4.
- We could use the code

```
lw $t0, AddrConstant4($s1)    # $t0 = constant 4
add $s3, $s3, $t0              # $s3 = $s3 + $t0 ($t0 == 4)
```

Constants

- Constant operands occur frequently
- If constants are stored in memory, performance will be effected
 - Slow to access memory
- Solution: use immediate

Immediate

- An alternative that avoids load instruction is to offer versions of the arithmetic instructions in which one operand is a constant
- This quick add instruction with one constant operand is called *add immediate* or *addi*
- Constant is stored within instruction

Example

- Add constant 4 to register \$s3. Assuming that \$s1 + AddrConstant4 is the memory address of the constant 4.
- We could use the code

```
lw $t0, AddrConstant4($s1)    # $t0 = constant 4
add $s3, $s3, $t0              # $s3 = $s3 + $t0 ($t0 == 4)
```

or:

Example

- Add the constant 4 to register \$s3. Assuming that \$s1 + AddrConstant4 is the memory address of the constant 4.
- We could use the code

```
lw $t0, AddrConstant4($s1)    # $t0 = constant 4
add $s3, $s3, $t0              # $s3 = $s3 + $t0 ($t0 == 4)
```

or:

```
addi $s3, $s3, 4
```


Constant Operands

- Occur frequently
- By including constants inside arithmetic instructions, operations are
 - much faster
 - use less energythan if constants were loaded from memory

Constant Zero

- MIPS dedicates a register `$zero` to be hard-wired to value `zero`
- It is register 0
- Cannot be overwritten

Move Operation

- Constant zero has another role, which is to simplify the instruction set by offering useful variations
- For example, **move** operation is just an add instruction where one operand is zero
 - Move (copy) data between registers

Example

- Copy data from register \$s1 to register \$t2.

Example

- Copy data from register \$s1 to register \$t2.

```
add $t2, $s1, $zero
```

Make Common Case Fast

- Using frequency to justify inclusions of constants
- Another example of the great idea of making the common case fast

Subtract Constants

- No subtract immediate instruction
- Since MIPS supports negative constants, no need for subtract immediate in MIPS

Example

- Subtract constant 1 from register \$s2
`addi $s2, $s2, -1`